UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

TRAS Nonlinear Controller

Berinde Codrin, Daisa Petru

Technical University of Cluj-Napoca

Faculty of Automation and Computer Science

Specialisation: Automation and Applied Informatics

Course: Control Engineering 2

Coordinating professors: Cristina Mureșan, Marcian Mihai

Year of study: 3

Date: June 2026

Contents

1 Project description **3**

2 Device description **3**
2.1 Physical quantities 4
2.2 Measured and controlled quantities 5

3 Physical modeling **5**
3.1 Vertical plane 6
3.2 Horizontal plane 8
3.3 Rotors rpm-input signal dependencies 9
3.4 Rotors force-rpm dependencies 10
3.5 The final model of the system 10

4 Identification **12**

5 Nonlinear controller design **12**
5.1 Theoretical aspects 12
5.2 Rotors subsystem 13
5.2.1 Process linearization 14
5.2.2 Preliminary discussion on controller type 14
5.2.3 Controller design 16
5.2.4 Steady state error discussion 18
5.3 Main subsystem 18
5.3.1 Process linearization 19
5.3.2 Controller design 20
5.3.3 Steady state error discussion 21
5.4 Final control structure 21

6 Implementation **23**

7 Performances **24**
7.1 Zero angle references 24
7.2 Zero angle references with external disturbances 25
7.3 Circles in space 25

8 Conclusions **25**

1 Project description

This is a one-semester long project that we, a team of two students at the Technical University of Cluj-Napoca, carried out for one of the most relevant – and some of us may say as well most beautiful – subject in our field of study: control engineering. Out of all the devices that were available in the laboratory for students to develop their projects on, we chose the Twin Rotor Aerodynamical System (TRAS), because it is a multiple-input-multiple-output, highly nonlinear system. It was our duty to perform the physical modeling, the identification, and, the ultimate goal, a discrete controller for the system. All the steps are presented in the following sections.

2 Device description

The Twin Rotor Aerodynamical System (TRAS) is an educational robot produced by INTECO Krakow. It can be controlled in real-time from a Matlab & Simulink environment. A schematic of the device can be seen in Figure 1.

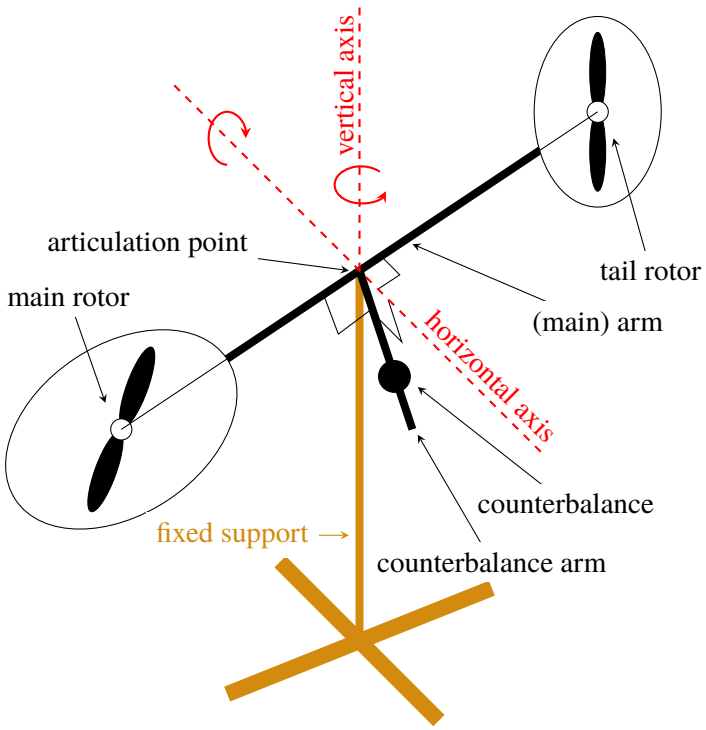


Figure 1: Schematic of the TRAS.

The device has a fixed support on which a mobile arm is linked at an articulation point. At one end of the arm there is the *main rotor* and at the other end the *tail rotor*. Understanding how the articulation and the movement of the arm work in 3D can be a bit tricky, therefore we will describe it in a more formal manner in what follows.

The articulation is actually composed of two separate articulations

1. the first one allows the arm to rotate, from any arbitrary initial position it might be, about the *vertical axis*, which is also marked on the figure. Note that a rotation *about* this axis means that the angle the arm forms *with* this axis is constant.

2. for any arbitrary position of the arm, we define the *horizontal axis* (also marked on the figure) as the horizontal line (so, parallel with the "table" on which the device stands) which crosses through the articulation point and is also perpendicular to the arm. Note that this axis, unlike the vertical one, is not fixed. It rotates together with the arm, if the arm performs a rotation about the vertical axis. The second articulation enables the rotation about this "instantaneous" horizontal axis that has just been described.

Obviously, the two types of rotation described above can be performed simultaneously.

The main rotor is coplanar with the horizontal axis at all times and the tail rotor is coplanar with the vertical axis at all times. Ignoring couplings and side effects, we can state that the main rotor, when in operation, rotates the arm **only** about the horizontal axis, while the tail rotor **only** about the vertical axis.

The device is also equipped with a counterbalance, which is a weight that can slide back and forth on a small arm (called "counterbalance arm") fixed to the main arm exactly at the articulation point. The counterbalance arm is perpendicular to the main arm. The vertical axis, the main arm and the counterbalance arm are three coplanar lines.

Every rotor has a plastic protective shield around it.

2.1 Physical quantities

The notations for various physical quantities (masses and lengths), proper to the device, that we will use throughout the documentation are the following

- m_m – the mass of the part of the arm between the articulation point and the main rotor;
- m_{mr} – the mass of the main rotor;
- m_{ms} – the mass of the main rotor protective shield;
- m_t – the mass of the part of the arm between the articulation point and the tail rotor;
- m_{tr} – the mass of the tail rotor;
- m_{ts} – the mass of the tail rotor protective shield;
- m_b – the mass of the counterbalance arm;
- m_{cb} – the mass of the counterbalance;
- l_m – the length of the main arm from the articulation point to the center of the main rotor;
- l_t – the length of the main arm from the articulation point to the center of the tail rotor;
- l_b – the length of the counterbalance arm;
- l_{cb} – the distance from the articulation point at which the counterbalance is attached on its rod;
- r_{ms} – the radius of the protective shield of the main rotor;
- r_{ts} – the radius of the protective shield of the tail rotor.

2.2 Measured and controlled quantities

The device measures with a sampling time interval of $t_s = 0.01$ s all of the following four quantities

- α_v – the angular position of the main arm in its rotation around the horizontal axis (in the "vertical plane", that is where the index comes from), also called *pitch* angle;
- ω_v – the angular speed of the main rotor (in rpm);
- α_h – the angular position of the main arm in its rotation around the vertical axis (in the "horizontal plane"), also called *azimuth* angle;
- ω_h – the angular speed of the tail rotor (in rpm).

The control signals are u_v and u_h, the main and tail rotor input voltages, which, as physical quantities, lie in the interval $[-12, 12]$V. However, the control is performed via a PWM signal, therefore we can consider that the control signal of the process is the duty cycle of this PWM, denoted as μ_v and μ_h, which belong to the interval $[-1, 1]$ (if μ is negative, it means a PWM signal of duty cycle $|\mu|$ but negative amplitude).

The purpose of this project is to control α_v and α_h via u_v and u_h (to follow a reference).

We will also denote

- Ω_v – the angular speed of the main arm around the horizontal axis, $\Omega_v = \frac{\mathrm{d}\alpha_v}{\mathrm{d}t}$;
- Ω_h – the angular speed of the main arm around the vertical axis, $\Omega_h = \frac{\mathrm{d}\alpha_h}{\mathrm{d}t}$.

but these quantities are not measured.

3 Physical modeling

To facilitate the design of a good nonlinear controller, an accurate model of the real system must be obtained first. For simplicity, however, the following phenomena have actually been completely ignored in the model (we considered that their influence is much smaller than that of the other contributors)

- the friction in the articulation – ignored because the friction is proportional with the velocity, however the entire project is realized to perform at low speeds;
- the friction with the air – for the same reason as the other friction above, as well as because this one is really, really small;
- the two couplings: between the tail rotor speed and its effect on the rotation around the horizontal axis (the other than the one it is intended for) and the coupling bewtween the main rotor speed and its effect on the rotation around the vertical axis – this has been observed in various experiments and can intuitively be understood by the reader by taking the following situation: let's say that we want to raise the main arm, but we do not want to modify its angle around the vertical axis at all. Therefore, we keep the tail rotor off and we turn on the main rotor in the positive direction (such that it exerts force downwards). Theoretically the arm will only rotate around the horizontal axis, which is mainly true but not entirely. When the main rotor rotates, one of the wings of the rotor will rotate clockwise with respect to the vertical axis, and the other will rotate counterclockwise. However, they are unequally distanced to the axis, which creates a non-zero momentum, resulting in a rotation around the vertical (unintended) axis.

This interaction has, however, been excluded from both the physical modeling and the controller design due to the difficulty of its identification (of finding the appropriate parameters) and of its inclusion in the nonlinear controller formula of the type we have used here.

We acknowledge that this is actually the most critical improvement that could be done to this project, should it be continued.

This being said, here is the mathematical description of the simplified process.

3.1 Vertical plane

By the vertical plane we mean the plane which contains both the main arm and the vertical axis. Note that since the arm rotates around the vertical axis, this plane rotates as well, together with the arm (it is a mobile plane). The projection of the device onto this plane at any arbitrary moment can be seen in Figure 2.

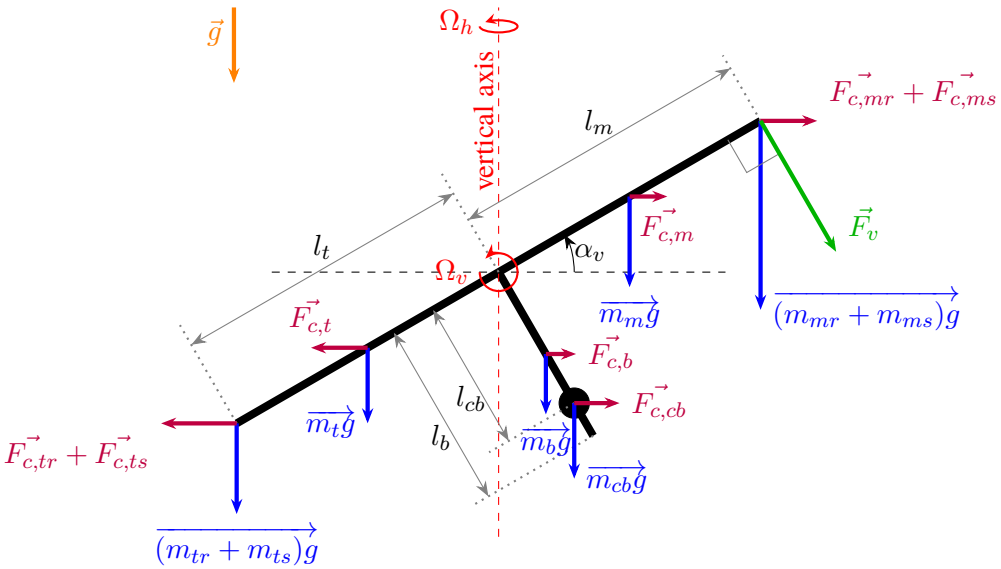


Figure 2: The TRAS in the vertical plane.

The virtual line from which the pitch angle α_v (vertical inclination) is measured is just a horizontal line that is coplanar with the arm (it is **not** the *horiozontal axis*, because that one is perpendicular to the plane we are currently viewing).

The forces depicted in blue are the gravitational forces and they together contribute to the gravitational torque which (by establishing the convention that the positive torques rotate trigonometrically, the same as the conventional direction of positive α_v, and the negatives one inverse trigonometrically) can be written as

$$\begin{aligned} M_G &= -(m_{mr}+m_{ms})gl_m\cos\alpha_v - m_m g\tfrac{l_m\cos\alpha_v}{2} + (m_{tr}+m_{ts})gl_t\cos\alpha_v \\ &\quad +m_t g\tfrac{l_t\cos\alpha_v}{2} - m_{cb}gl_{cb}\sin\alpha_v - m_b g\tfrac{l_b\sin\alpha_v}{2} \end{aligned}$$

which by denoting $A=\left(m_{tr}+m_{ts+\frac{m_t}{2}}\right)l_t-\left(m_{mr}+m_{ms}+\frac{m_m}{2}\right)l_m$ and $C=m_b\frac{l_b}{2}+m_{cb}l_{cb}$ results in

$$M_G = g\left[A\cos\alpha_v - C\sin\alpha_v\right] \tag{1}$$

Then, the main rotor generates a torque that can be expressed as

$$M_{R,v} = l_m F_v(\omega_v) \tag{2}$$

where F_v is a function that describes the dependency of the force exerted by the rotor, on its rpm. This function will be described later. Note that the force exerted by the rotor is always perpendicular to the arm.

Next, in purple we have the centrifugal forces caused by the rotation of the arm around the **vertical** axis (rotation from the other plane). They are expressed as

$$F_{c,mr} + F_{c,ms} = (m_{mr} + m_{ms})\Omega_h^2 l_m \cos\alpha_v$$

$$F_{c,m} = m_m \Omega_h^2 \frac{l_m}{2} \cos\alpha_v$$

$$F_{c,tr} + F_{c,ts} = (m_{tr} + m_{ts})\Omega_h^2 l_t \cos\alpha_v$$

$$F_{c,t} = m_t \Omega_h^2 \frac{l_t}{2} \cos\alpha_v$$

$$F_{c,cb} = m_{cb} \Omega_h^2 l_{cb} \sin\alpha_v$$

$$F_{c,b} = m_b \Omega_h^2 \frac{l_b}{2} \sin\alpha_v$$

The torque produced by these forces can be written as

$$\begin{aligned} M_{CF} \;=\; & -(F_{c,mr} + F_{c,ms}) l_m \sin\alpha_v - F_{c,m} \tfrac{l_m}{2} \sin\alpha_v \\ & -(F_{c,tr} + F_{c,ts}) l_t \sin\alpha_v - F_{c,t} \tfrac{l_t}{2} \sin\alpha_v \\ & +F_{c,cb} l_{cb} \cos\alpha_v + F_{c,b} \tfrac{l_b}{2} \cos\alpha_v \end{aligned}$$

which is

$$\begin{aligned} M_{CF} \;=\; & -\tfrac{\Omega_h^2}{2} \left[\left(m_{mr} + m_{ms} + \tfrac{m_m}{4}\right) l_m^2 + \left(m_{tr} + m_{ts} + \tfrac{m_t}{4}\right) l_t^2 \right. \\ & \left. -m_{cb} l_{cb}^2 - \tfrac{m_b}{4} l_b^2 \right] \sin(2\alpha_v) \end{aligned}$$

and by further denoting $L = \left(m_{mr} + m_{ms} + \frac{m_m}{4}\right) l_m^2 + \left(m_{tr} + m_{ts} + \frac{m_t}{4}\right) l_t^2 - m_{cb} l_{cb}^2 - \frac{m_b}{4} l_b^2$ we obtain

$$M_{CF} = -\frac{\Omega_h^2}{2} L \sin(2\alpha_v) \tag{3}$$

Therefore, from equations (1), (2) and (3), we obtain the total torque in the vertical plane as

$$M_v = M_G + M_{R,v} + M_{CF} = g(A\cos\alpha_v - C\sin\alpha_v) + l_m F_v(\omega_v) - \frac{\Omega_h^2}{2} L \sin(2\alpha_v) \tag{4}$$

Now, in order to be able to write Newton's law for this movement, we need to express the total moment of inertia of the system in the vertical plane. All the 8 elements which have masses contribute to this moment.

$$\begin{aligned} J_v \;=\; & m_{mr} l_m^2 + m_{ms}(l_m^2 + r_{ms}^2) + m_m \tfrac{l_m^2}{3} \\ & +m_{tr} l_t^2 + m_{ts}(l_t^2 + r_{ts}^2) + m_t \tfrac{l_t^2}{3} \\ & +m_b \tfrac{l_b^2}{3} + m_{cb} l_{cb}^2 \end{aligned}$$

Finally, we can write Newton's law for the rotational movement in the vertical plane as

$$\frac{\mathrm{d}\Omega_v^2}{\mathrm{d}t} = \frac{M_v}{J_v} \tag{5}$$

3.2 Horizontal plane

By the horizontal plane we mean the plane to which the vertical axis is always perpendicular and contains the horizontal axis. Unlike the vertical plane, this one is fixed, it is not dependent on the current position of the device. A projection of the system onto this plane, seen from above, can be viewed in Figure 3.

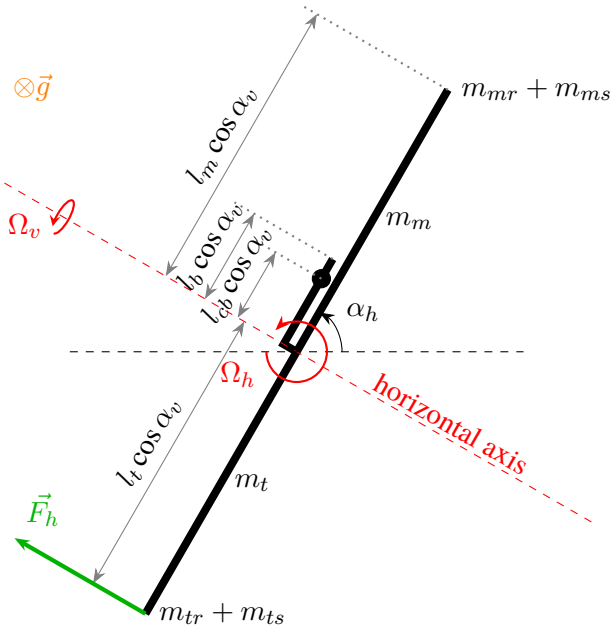


Figure 3: The TRAS in the horizontal plane.

The virtual line from which the azimuth angle α_h (horizontal inclination) is measured is just a reference horizontal line. It is not that important exactly where this line is, because it depends on the actual position where the device has started (in that position, it will measure $\alpha_h = 0$), but it is very important that this line is perfectly horizontal (like the horizontal axis), but unlike the horizontal axis, it remains fixed throughout the entire execution of the program.

Here the number of forces is considerably smaller than for the vertical plane, because no gravitational force can ever rotate the system in this plane (since the gravitational acceleration is perfectly perpendicular to it, as shown in the figure) and also all the centrifugal forces generated by the rotation in the other plane are existent but are always on the direction that passes through the articulation point, so their torques is zero all the time. However, it is very important to correctly visualize the system in this plane. Note that the lengths of the arms, which are actually *apparent* lengths for this plane (projections) are not l_m and l_t, but rather $l_m \cos\alpha_v$ and $l_t \cos\alpha_v$, because the entire system is considered to be rotated at an arbitrary position α_v in the other plane as well! Therefore, for this plane, the two sides of the main arm will have shorter apparent lengths.

The only force that remains for this plane is the tail rotor force. The total horizontal torque of the system can be written as

$$M_h = M_{R,h} = F_h(\omega_h) l_t \cos\alpha_v \tag{6}$$

where F_h is a function that describes the dependency of the force exerted by the tail rotor, on its rpm.

The total moment of inertia of the system in the horizontal plane is given by all the 8 elements which have masses: the tail rotor, the tail rotor shield, the tail arm, the main rotor, the main rotor shield, the main rotor arm, the counterbalance and the counterbalance arm. We can write the horizontal moment of

inertia as

$$\begin{aligned} J_h &= m_{mr}(l_m \cos\alpha_v)^2 + m_{ms}\left[(l_m \cos\alpha_v)^2 + r_{ms}^2\right] + m_m \tfrac{(l_m \cos\alpha_v)^2}{3} \\ &\quad + m_{tr}(l_t \cos\alpha_v)^2 + m_{ts}\left[(l_t \cos\alpha_v)^2 + r_{ts}^2\right] + m_t \tfrac{(l_t \cos\alpha_v)^2}{3} \\ &\quad + m_{cb}(l_{cb} \sin\alpha_v)^2 + m_b \tfrac{(l_b \sin\alpha_v)^2}{3} \end{aligned}$$

which by denoting $D = \left(m_{mr} + m_{ms} + \frac{m_m}{3}\right) l_m^2$, $E = \left(m_{tr} + m_{ts} + \frac{m_t}{3}\right) l_t^2$ and $F = m_{cb} l_{cb}^2 + m_b \frac{l_b^2}{3}$ can be written as

$$J_h = D\cos^2\alpha_v + E\sin^2\alpha_v + F \tag{7}$$

and finally Newton's law of motion in the horizontal plane

$$\frac{\mathrm{d}\Omega_h^2}{\mathrm{d}t} = \frac{M_h}{J_h} \tag{8}$$

3.3 Rotors rpm-input signal dependencies

We would also like a good model of the rotors, namely a dependency of their angular velocities, ω_v and ω_h, on the control signals μ_v and μ_h. Both rotors have the same model, the only difference being in the numerical values of the parameters, which is why we drop the subscripts v and h below.

This system is a DC motor with a rotor load, modelled as $C_d\omega^2$:

$$\begin{cases} L\frac{\mathrm{d}i}{\mathrm{d}t} = V - Ri - K\omega \\ J\frac{\mathrm{d}\omega}{\mathrm{d}t} = Ki - B\omega - C_d\omega^2 \end{cases} \tag{9}$$

The first equation is the electrical one, with L, i, R being the armature inductance, current, resistance and K, V being the back emf constant and the input voltage. The second equation is the mechanical one, with J being the moment of inertia, B viscous friction coefficient and C_d the aerodynamic drag coefficient. Since the time constants of the first equation are significantly smaller than those of the other one, we assume that $\frac{\mathrm{d}i}{\mathrm{d}t}$ is zero, which leads to a simplified single equation:

$$J\frac{\mathrm{d}\omega}{\mathrm{d}t} = \frac{K}{R}V - \left(\frac{K^2}{R} + B\right)\omega - C_d\omega^2 \tag{10}$$

This model assumes the electrical part is always connected, even when the input is off, which would mean quick braking when the input is zero, which is not what we observe. Thus, we deduce that the motor is disconnected from the electrical part completely when the PWM input is on the off phase. Thus, for a signed duty cycle $\mu \in [-1, 1]$ and the constant power supply E, we average the two cases: when the power is on and when the power is off:

$$J\frac{\mathrm{d}\omega}{\mathrm{d}t} = |\mu|\left(\frac{K}{R}\,\mathrm{sign}(\mu)E - \left(\frac{K^2}{R} + B\right)\omega - C_d\omega|\omega|\right) + (1 - |\mu|)(-B\omega - C_d\omega|\omega|) \tag{11}$$

By reducing the terms we obtain the lumped parameter model shown below

$$\frac{\mathrm{d}\omega}{\mathrm{d}t} = \theta_1\mu - \omega(\theta_2|\mu| + \theta_3) - \theta_4\omega|\omega| \tag{12}$$

Notice that we don't need to work with the individual paremeters (like R, K) since they are multiplied together and don't have an independent effect on the result. This approach of lumping the parameters was also used before and it is the right approach for grey box identification.

3.4 Rotors force-rpm dependencies

For any of the two rotors, the dependency of the force they exert on the rpm generally does not depend on past values, therefore it can be described using non-differential equations.

Such a dependency is given in the user manual of the device and was taken as such in this project, because identification experiments are more difficult to perform for this specific dependency.

The main rotor $F_v(\omega_v)$ dependency is given by

$$F_v(\omega_v) = -1.8 \cdot 10^{-18}\omega_v^5 - 7.8 \cdot 10^{-16}\omega_v^4 + 4.1 \cdot 10^{-11}\omega_v^3 + 2.7 \cdot 10^{-8}\omega_v^2 + 3.5 \cdot 10^{-5}\omega_v$$

and can be viewed in Figure 4.

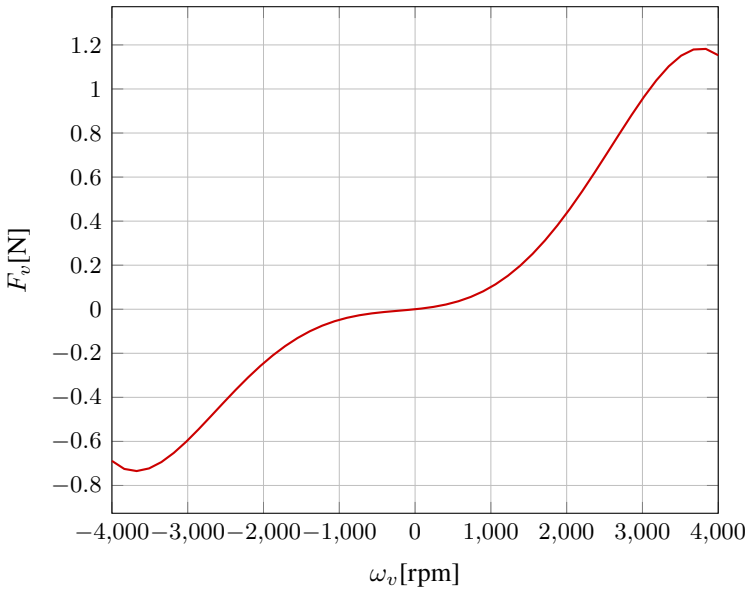


Figure 4: Main rotor force-rpm dependency

The tail rotor $F_h(\omega_h)$ dependency is given by

$$F_h(\omega_h) = -2.6 \cdot 10^{-20}\omega_h^5 + 4.1 \cdot 10^{-17}\omega_h^4 + 3.2 \cdot 10^{-12}\omega_h^3 - 7.3 \cdot 10^{-9}\omega_h^2 + 2.1 \cdot 10^{-5}\omega_h$$

and can be viewed in Figure 5.

3.5 The final model of the system

Putting together everything that was discussed above, we can now write the state-space system of equations that govern the dynamics of this system. The vector of state variables is

$$\mathbf{x} = \begin{bmatrix} \Omega_v & \alpha_v & \omega_v & \Omega_h & \alpha_h & \omega_h \end{bmatrix}^\top$$

the vector of output variables is

$$\mathbf{y} = \begin{bmatrix} \alpha_v \\ \alpha_h \end{bmatrix}$$

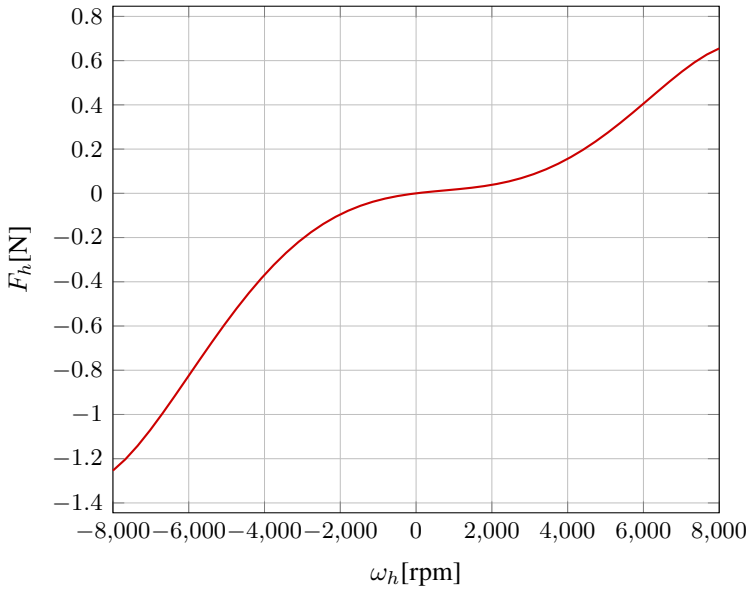


Figure 5: Tail rotor force-rpm dependency

and the vector of input variables

$$\mathbf{u} = \begin{bmatrix} \mu_v \\ \mu_h \end{bmatrix}$$

From equations (4), (5), (6), (8) and (12) we get

$$\begin{cases} \dfrac{\mathrm{d}\Omega_v}{\mathrm{d}t} = \dfrac{1}{J_v}\left[g(A\cos\alpha_v - C\sin\alpha_v) + l_m F_v(\omega_v) - \dfrac{\Omega_h^2}{2}L\sin(2\alpha_v)\right] \\ \dfrac{\mathrm{d}\alpha_v}{\mathrm{d}t} = \Omega_v \\ \dfrac{\mathrm{d}\omega_v}{\mathrm{d}t} = \theta_{1v}\mu_v - \omega_v(\theta_{2v}|\mu_v| + \theta_{3v}) - \theta_{4v}\omega_v|\omega_v| \\ \dfrac{\mathrm{d}\Omega_h}{\mathrm{d}t} = \dfrac{F_h(\omega_h)l_t\cos\alpha_v}{D\cos^2\alpha_v + E\sin^2\alpha_v + F} \\ \dfrac{\mathrm{d}\alpha_h}{\mathrm{d}t} = \Omega_h \\ \dfrac{\mathrm{d}\omega_h}{\mathrm{d}t} = \theta_{1h}\mu_h - \omega_h(\theta_{2h}|\mu_h| + \theta_{3h}) - \theta_{4h}\omega_h|\omega_h| \end{cases} \tag{13}$$

where $\theta_v = [\ \theta_{1v} \quad \theta_{2v} \quad \theta_{3v} \quad \theta_{4v}\ ]^\top$, $\theta_h = [\ \theta_{1h} \quad \theta_{2h} \quad \theta_{3h} \quad \theta_{4h}\ ]^\top$, A, C, D, E, F, L, J_v, g, l_m and l_t are all parameters that have been discussed before. The dependencies $F_v(\omega_v)$ and $F_h(\omega_h)$ have been kept with this notation here so that we don't have to write those very long polynomials.

This system of differential equations – highly nonlinear, dare we say – may seem overwhelming at first sight, but it is actually not. The first and fourth equations are Newton's law of motion applied on both planes. The second and fifth equations are just the definitions of angular speeds as derivatives of angular positions. The third and sixth equations are the dependencies of the rotor's forces on their rpm's,

and it can be observed that in their right side no other state variables appear than themselves (the rpm's evolutions in time is, as expected, dependent only on the input variables and their own past values, but independent from the other states).

Good knowledge of the process is a key element for a performant controller. In the next two sections, we will discuss the identification procedures of some of the parameters in (13) and next, the design of the nonlinear controller, which starts exactly from the model.

4 Identification

The performance of the controller depends on the accuracy of the model, even more so for nonlinear unforgiving systems like this one. Obtaining a good model comes down to two things: the amount of detail captured by the dynamics (the equations themselves, as seen before) and the numerical value of the parameters, such that the behavior of the model is as close to that of the system. This process is called *grey box identification*, as opposed to *black box identification*, where not only the numerical values of the parameters are to be found, but also the equations themselves (though they have to obey a certain structure as well, for example, linear ODEs).

The procedure followed is standard (see https://www.mathworks.com/help/optim/ug/lsqnonlin.html for a formal description): we prepare an input signal which we feed into the real system and we record the output. Then we "train" the model to respond as similarly as possible to the same inputs. Despite the computer doing all this computation, the identification is not trivial, due to the amount of information required to model such a complex system. The greatest enemy is the real world; in other words, all models are far from perfect, including ours. Trying to identify all the parameters with one input signal would be delusional, for many reasons.

The solution to this first problem, the big number of parameters, is to fix most of them and systematically free small independent subsets to be identified. This works if we can excite only parts of the dynamics (i.e. either the fixed parameters are known to be good or no matter what values they have, they don't influence the response to that specific input), which it turns out we can. The simplest are the rotors, because we have both input and feedback directly from this subsystem. Next we identify the gravity related parameters by setting both inputs to zero and measuring the response from non-zero initial conditions, allowing the system to reach equilibirium under the influence of gravity. This leads to identification of A and C. We have initially prepared also strategies for isolating other parameters, but the model mismatch as well as the insufficient information from the input signal hindered our efforts, leaving us with the default values (from the device manual) for the rest of the parameters. Note, however, that there was a substantial improvement in terms of responsiveness even only with these few identified values.

Now a few words on the design of the input signals. They have to be informative enough, to excite the system sufficiently for the number of parameters we try to identify. However, applying white noise or a PRBS won't work here, due to nonlinearities. For this reason, we used a sequence of randomized steps (the amplitude and duration of each step was a sample from some normal distribution) as input for the rotor identification, which intuitively resembles a little the control signal we are going to apply and also moves the rotor over the entire range of interest.

5 Nonlinear controller design

5.1 Theoretical aspects

For designing the controller of a nonlinear system, we have applied the **feedback linearization** technique. As can be seen in (13), this system is very difficult to operate with, given the fact that state variables appear

squared, in absolute values, or even at higher powers (in the F_v and F_h polynomials). So, in order to still be able to make a controller, a good one, that follows certain performances, we can do a little "trick", as explained below. The general structure of a control loop (nonlinear, in our case) is presented in Figure 6.

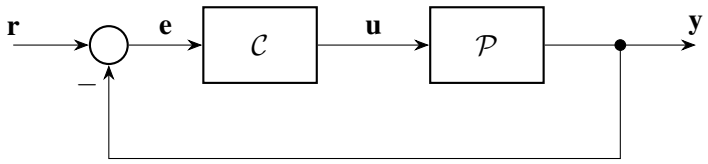


Figure 6: The general structure of a control loop

Note that in Figure 6, **r**, **e**, **u** and **y** are multiplexed signals (vectors), because this is a MIMO (multiple-input-multiple-output) system.

The general approach of this technique is to try to separate (because we cannot work with how it is now) the initial process $\mathcal{P}$ into two separate blocks: $\mathcal{P}_{\text{nonlinear}}$ which must mandatory come first and include all the nonlinearities of the process, and $\mathcal{P}_{\text{linear}}$, the linear part that is left, which comes second. Now, we actually know how to design a controller for the second (linear) part, but what do we do with the first one? We separate the controller as well into two blocks: $\mathcal{C}_{\text{linear}}$, which comes first, and which represents the controller for $\mathcal{P}_{\text{linear}}$, and $\mathcal{P}^{-1}_{\text{nonlinear}}$, which represents the perfect cancelation of the nonlinear part. The "linearized" loop can be observed in Figure 7.

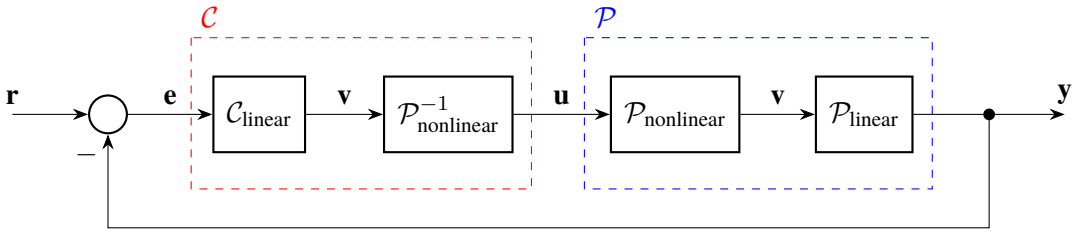


Figure 7: The "linearized" structure of a control loop

As of now, the nonlinear components of the process completely cancel out, and we are left with a linear process and a controller for it, which we know well how to design by applying basic linear control theory. Note that the new control signal generated by our linear controller (which is then transformed into the old control signal **u** by the cancelation part $\mathcal{P}^{-1}_{\text{nonlinear}}$ and then back into itself by the nonlinear part of the process) is denoted as **v** and is called *virtual control signal*, because it actually never comes into play as a real physical signal.

Now, in its simplicity, this solution may seem like a magical tool to use. However, in many situations, it is not applicable due to the following fundamental restriction. For the nonlinear part of the process to be perfectly invertible inside the controller, it has to be composed **solely** out of **algebraic equations**, not differential ones! If such a separation can be performed, then chances are very high that a nonlinear controller using the feedback linearization technique can be designed. And it is the lucky case also for our nonlinear process.

5.2 Rotors subsystem

For the beginning, look at the system of equations (13) and take apart the third and sixth equations, which describe the subsystem of the rotors. This can be considered a subsystem because its dynamics is completely separated from the rest of the system (in these differential equations, no other state variables

of the process appear). We will first design a controller for this subsystem of rotors, using the feedback linearization technique. The input signals are $\mathbf{u}_{\text{rotor}} = \begin{bmatrix} \mu_v & \mu_h \end{bmatrix}^\top$, the states and the output signals are $\mathbf{x}_{\text{rotor}} = \mathbf{y}_{\text{rotor}} = \begin{bmatrix} \omega_v & \omega_h \end{bmatrix}^\top$.

$$\begin{cases} \dfrac{\mathrm{d}\omega_v}{\mathrm{d}t} = \theta_{1v}\mu_v - \omega_v(\theta_{2v}|\mu_v| + \theta_{3v}) - \theta_{4v}\omega_v|\omega_v| \\ \dfrac{\mathrm{d}\omega_h}{\mathrm{d}t} = \theta_{1h}\mu_h - \omega_h(\theta_{2h}|\mu_h| + \theta_{3h}) - \theta_{4h}\omega_h|\omega_h| \end{cases} \tag{14}$$

It is also true that these two equations are completely independent from each other and they differ only in the values of the parameters, so we will design a generic controller that will later be applied to both rotors, for different values of θs. The generic model of a rotor is therefore

$$\frac{\mathrm{d}\omega}{\mathrm{d}t} = \theta_1\mu - \omega(\theta_2|\mu| + \theta_3) - \theta_4\omega|\omega|$$

with the input signal μ, and the only state and, at the same time, output signal ω.

Again, this is a nonlinear system in essence, because of the absolute value and of the fact that ω appears at power 2 in the last term.

5.2.1 Process linearization

Now let us try to perform a separation of this nonlinear process into a nonlinear algebraic part, and a linear differential one, as discussed in the theoretical section. And such a separation is possible, by denoting $v_{\text{rotor}} = \theta_1\mu - \omega(\theta_2|\mu| + \theta_3) - \theta_4\omega|\omega|$, the virtual signal equal to the entire right side of the equation. By doing this, we obtain the following description of the system

$$\begin{cases} v_{\text{rotor}} = \theta_1\mu - \omega(\theta_2|\mu| + \theta_3) - \theta_4\omega|\omega| & (15.1) \\ \dfrac{\mathrm{d}\omega}{\mathrm{d}t} = v_{\text{rotor}} & (15.2) \end{cases}$$

where equation (15.1) represents $\mathcal{P}_{\text{nonlinear}}$ and (15.2) represents $\mathcal{P}_{\text{linear}}$.

Now we are left only with the problem of designing a good linear controller for the system described by (15.2), where the input (control) signal is v_{rotor} (virtual). The controller, in the $\mathcal{P}^{-1}_{\text{nonlinear}}$, will then compute the real control signal μ by extracting it from (15.1), which is possible due to the fact that ω is measured by the sensors, so the controller knows its current value.

5.2.2 Preliminary discussion on controller type

We have the following linearized process given by (15.2)

$$\frac{\mathrm{d}\omega}{\mathrm{d}t} = v_{\text{rotor}} \Longrightarrow \omega(t) = \int_0^t v_{\text{rotor}}(\tau)\mathrm{d}\tau \tag{16}$$

which is a simple integrator $P(s) = \frac{1}{s}$. Now, it may seem obvious that since we already have this integrator, the steady-state error of the controlled process will be zero, no matter what the transfer function of the controller is. However, this is not true, and in what follows we will prove it.

Theorem 5.1. *Given a system controlled via the feedback linearization technique with the linear part of the process described by $\mathcal{P}_{linear} = P(s)$ that integrates its input (virtual control) signal n times, $n \geq 1$. The steady state error ε_{ss} is zero if and only if the linear part of the controller $\mathcal{C}_{linear} = C(s)$ contains at least one integrator.*

Proof. Assume an arbitrary linear process resulted from the linearization procedure: $P(s)$. Assume the reference signal $R(s) = \frac{1}{s}$ (unit step reference), the error signal is $E(s) = R(s) - Y(s)$ where $Y(s)$ is the output signal of the process, and the linear controller transfer function is $C(s) = \frac{V(s)}{E(s)}$, where $V(s)$ is the virtual control signal.

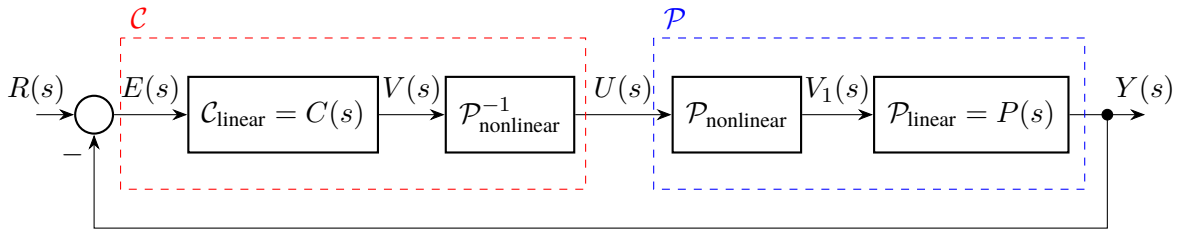


Figure 8: The assumed control structure.

The controller receives the signal $E(s)$ at its input and generates $V(s) = E(s)C(s)$. Then, the controller also computes, based on its knowledge, the real control signal $U(s)$ out of $V(s)$. The process $\mathcal{P}$ is virtually decomposed into a nonlinear (first) part $\mathcal{P}_{\text{nonlinear}}$, and a linear (second) part $P(s)$. Now, the problem here is that given $U(s)$ the input to the process, its nonlinear part will not turn it back into exactly $V(s)$, the virtual control signal from the controller, but into some other, slightly different, $V_1(s)$. This is due to the fact that the controller only knows an approximation of the real process, and the inversion within the controller, $\mathcal{P}^{-1}_{\text{nonlinear}}$, does not perfectly invert $\mathcal{P}_{\text{nonlinear}}$. There will always be a difference between $V(s)$ and $V_1(s)$, which we can formally write as

$$V_1(s) = V(s) + L(s)$$

where $L(s)$ is some residue given by imperfections in the modeling. We do not know much about this function $L(s)$, but we expect that even at steady state, its value is non-zero, which translates to $\lim_{t\to\infty} l(t) = l_{ss} \neq 0$, which from the final value theorem is

$$\lim_{s\to 0} sL(s) = l_{ss} \neq 0 \tag{17}$$

Then, the linear part of the process will transform its input $V_1(s)$ into the output $Y(s)$, given by

$$Y(s) = P(s)V_1(s) = P(s)\left[V(s) + L(s)\right]$$

but $Y(s) = R(s) - E(s)$, so

$$R(s) - E(s) = P(s)\left[C(s)E(s) + L(s)\right]$$

$$E(s)\left[1 + P(s)C(s)\right] = R(s) - P(s)L(s)$$

from which

$$E(s) = \frac{R(s) - P(s)L(s)}{1 + P(s)C(s)} \tag{18}$$

Here, as per the hypothesis, the process contains n integrators, $n \geq 1$. Therefore, it can be written as $P(s) = s^{-n}P'(s)$, where $P'(s)$ is the integrator-free part and therefore $\lim_{s\to 0} P'(s) = p'_{ss}$ is a real non-zero number. Also, we know that $R(s) = \frac{1}{s}$. We obtain the value of the steady-state error by applying the final value theorem to equation (18)

$$\varepsilon_{ss} = \lim_{t\to\infty} \varepsilon(t) = \lim_{s\to 0} sE(s) = \lim_{s\to 0} s\frac{\frac{1}{s} - s^{-n}P'(s)L(s)}{1 + s^{-n}P'(s)C(s)} = \lim_{s\to 0} \frac{1 - s^{-n+1}P'(s)L(s)}{1 + s^{-n}P'(s)C(s)}$$

$$\varepsilon_{ss} = \lim_{s \to 0} \frac{s^n - P'(s)sL(s)}{s^n + P'(s)C(s)} \tag{19}$$

First, we prove the direct implication of the theorem. We will prove its negated form, that is: if the controller does not contain any integrator, it results that the steady state error is non-zero. In equation (19) above, assume $C(s)$ does not contain any integrator. It means that there exists $\lim_{s \to 0} C(s) = c_{ss}$ a real number. In this situation we obtain

$$\varepsilon_{ss} = \frac{0 - p'_{ss} l_{ss}}{0 + p'_{ss} c_{ss}} = -\frac{l_{ss}}{c_{ss}}$$

which is non-zero. Therefore, we have proved that even if the process contains integrators (no matter how many), the steady state error is non-zero if there exists any imperfection in the modeling of the process, that is $l_{ss} \neq 0$ (which obviously exists all the time). We can reduce the steady state error by decreasing l_{ss}, which means a better model, or by increasing c_{ss}, the gain of the controller, which translates to a more aggressive control – good in theory but often not feasible in the real world due to actuator limitations.

Now let us prove the reverse implication. If we have one integrator in the controller, which means that the error signal is integrated once within the control law, we can write the controller transfer function as

$$C(s) = K_i \frac{1}{s} + M(s) \tag{20}$$

where $M(s)$ is some other component that does not have integrators. The controller transfer function becomes

$$C(s) = \frac{K_i + sM(s)}{s} = \frac{1}{s} C'(s)$$

where $C'(s)$ is the integrator-free part of the controller and $\lim_{s \to 0} C'(s) = c'_{ss}$ exists and is a real non-zero number. Then we rewrite (19) as

$$\varepsilon_{ss} = \lim_{s \to 0} \frac{s^n - P'(s)sL(s)}{s^n + s^{-1}P'(s)C'(s)} = \lim_{s \to 0} s \frac{s^n - P'(s)sL(s)}{s^{n+1} + P'(s)C'(s)} = 0 \cdot \frac{0 - p'_{ss} l_{ss}}{0 + p'_{ss} c'_{ss}} = 0$$

■

We conclude that by introducing an integrator into the controller we have reduced the steady-state error to an absolute zero, no matter the imperfections in the modeling. Of course, this does not mean that we can now allow ourselves to model the real process very badly. Only the steady state error is now guaranteed to drop to zero; a bad modeling may lead to other serious inconveniences, such as high oscillations or even instability.

5.2.3 Controller design

Coming back to our problem, we have to design a linear controller for the linearized process

$$P(s) = \frac{1}{s}$$

We will not apply a method like Guillemin-Truxal here, because given this specific process transfer function, the resulted controller will not have an integrator, and, as per Theorem 5.1, it will not exhibit zero steady-state error. Instead, let us start our design from the equation of the error. We look carefully at two aspects: first, in the model of the system, the highest order of a derivative is 1, so also in the control law we want to have at most a first order derivative. And second, we have just proved that the error can

be reduced to zero at steady state only if it is integrated here. Therefore, we impose that the error in thys subsystem, $\varepsilon_{\text{rotor}}$, obeys the following relation

$$\frac{\mathrm{d}\varepsilon_{\text{rotor}}(t)}{\mathrm{d}t} + K_p\varepsilon_{\text{rotor}}(t) + K_i\int_0^t \varepsilon_{\text{rotor}}(\tau)\mathrm{d}\tau = 0 \tag{21}$$

Equation (21) may seem to come out of nowhere, but, in fact, it is a very handy way to impose certain performance specifications on the system. Let us see how. First, turn it into a differential equation

$$\frac{\mathrm{d}^2\varepsilon_{\text{rotor}}(t)}{\mathrm{d}t^2} + K_p\frac{\mathrm{d}\varepsilon_{\text{rotor}}(t)}{\mathrm{d}t} + K_i\varepsilon_{\text{rotor}}(t) = 0$$

which has the characteristic equation

$$\lambda^2 + K_p\lambda + K_i = 0$$

and by comparing it to the standard equation of a second-order system

$$\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2 = 0$$

it follows that

$$\begin{cases} K_p = 2\zeta\omega_n \\ \\ K_i = \omega_n^2 \end{cases} \tag{22}$$

meaning that we can tune K_p and K_i based on performance specifications such as overshoot σ and settling time t_s as

$$\begin{cases} \zeta = \dfrac{|\ln\sigma|}{\sqrt{\ln^2\sigma + \pi^2}} \\ \\ \omega_n = \dfrac{4}{\zeta t_s} \end{cases} \tag{23}$$

The values of ζ and ω_n obtained in (23) from the performance specifications are then replaced in (22) to obtain the controller parameters K_p and K_i.

Now let us actually derive the control law. By acknowledging that

$$\frac{\mathrm{d}\varepsilon_{\text{rotor}}(t)}{\mathrm{d}t} = \frac{\mathrm{d}}{\mathrm{d}t}(\omega_{\text{ref}}(t) - \omega(t)) \overset{(16)}{=} \frac{\mathrm{d}\omega_{\text{ref}}(t)}{\mathrm{d}t} - v_{\text{rotor}}$$

where ω_{ref} is the reference of the rotors subsystem and v the virtual control signal, we can rewrite the imposed equation (21) for the error as

$$\frac{\mathrm{d}\omega_{\text{ref}}(t)}{\mathrm{d}t} - v_{\text{rotor}}(t) + K_p\varepsilon_{\text{rotor}}(t) + K_i\int_0^t \varepsilon_{\text{rotor}}(\tau)\mathrm{d}\tau = 0$$

which is rearranged into exactly the control law of the controller

$$v_{\text{rotor}}(t) = \frac{\mathrm{d}\omega_{\text{ref}}(t)}{\mathrm{d}t} + K_p\varepsilon_{\text{rotor}}(t) + K_i\int_0^t \varepsilon_{\text{rotor}}(\tau)\mathrm{d}\tau \tag{24}$$

which is a PI (proportional-integral) controller *with feedforward*, because the controller does not only manipulate the error, but also the reference, in its law. The final schema of the controlled subsystem of the rotors (for one rotor) can be seen in Figure 9, based on the model in Figure 7. The same controller design is applied to both rotors, of course, for their own parameters θ_v and θ_h.

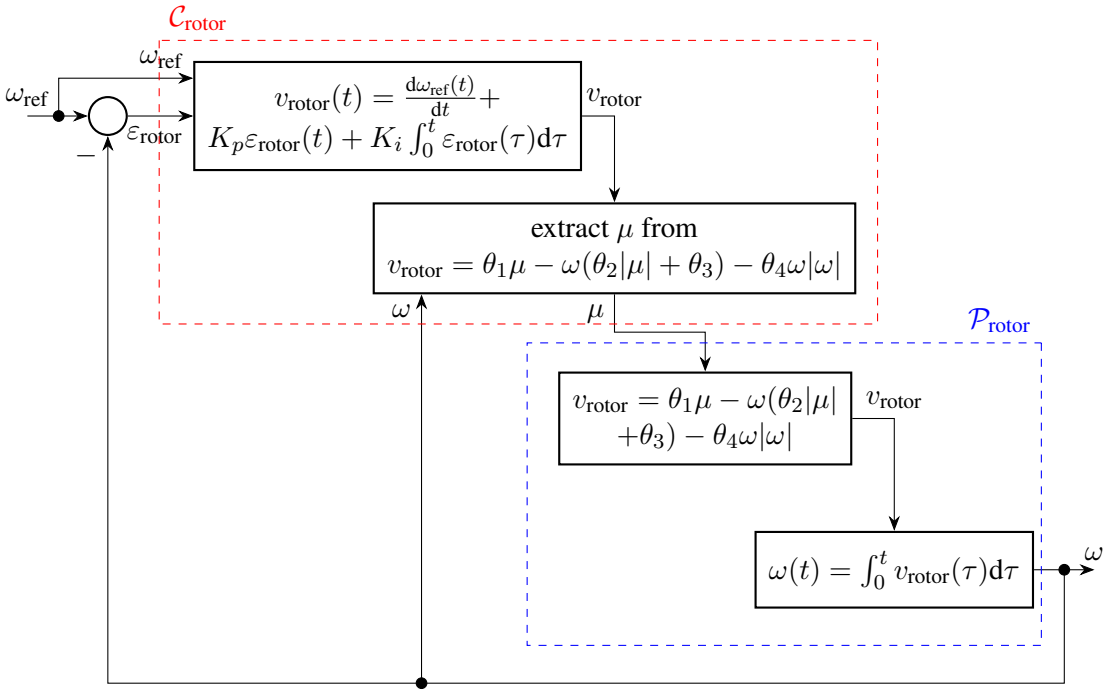


Figure 9: The controlled subsystem of the rotors.

5.2.4 Steady state error discussion

We have achieved the control law (24). Its Laplace transform is

$$V(s) = sR(s) + K_pE(s) + K_i\frac{1}{s}E(s)$$

from which the controller transfer function is

$$C(s) = \frac{V(s)}{E(s)} = K_i\frac{1}{s} + K_p + s\frac{R(s)}{E(s)}$$

which is of the form (20), if we denote $M(s) = K_p + s\frac{R(s)}{E(s)}$. By applying Theorem 5.1 we conclude that the steady state error is guaranteed to be zero.

5.3 Main subsystem

Now that we have trained our skills of nonlinear control design on the subsystem of the rotors, we are prepared to take the ultimate challenge and design a good controller for the main process. The fact that we separated the entire process into a subsystem and *what is left* was a very helpful decision in the sense that the mathematics got much easier and enabled us to design the controller only for that subsystem. Now, the good news is that *what is left* can also be viewed as an independent subsystem. If from (13) we take

the first, second, fourth and fifth equations, which is

$$\begin{cases} \dfrac{\mathrm{d}\Omega_v}{\mathrm{d}t} = \dfrac{1}{J_v}\left[g(A\cos\alpha_v - C\sin\alpha_v) + l_m F_v(\omega_v) - \dfrac{\Omega_h^2}{2}L\sin(2\alpha_v)\right] \\ \dfrac{\mathrm{d}\alpha_v}{\mathrm{d}t} = \Omega_v \\ \dfrac{\mathrm{d}\Omega_h}{\mathrm{d}t} = \dfrac{F_h(\omega_h) l_t \cos\alpha_v}{D\cos^2\alpha_v + E\sin^2\alpha_v + F} \\ \dfrac{\mathrm{d}\alpha_h}{\mathrm{d}t} = \Omega_h \end{cases} \tag{25}$$

and look closer, we can claim that if we consider $\mathbf{y}_{\text{main}} = \begin{bmatrix} \alpha_v & \alpha_h \end{bmatrix}^\top$ the output, $\mathbf{x}_{\text{main}} = \begin{bmatrix} \Omega_v & \alpha_v & \Omega_h & \alpha_h \end{bmatrix}^\top$ the states and $\mathbf{u}_{\text{main}} = \begin{bmatrix} \omega_v & \omega_h \end{bmatrix}^\top$ the input, then this is also a completely independent subsystem from that of the rotors, which we will call *the main subsystem*.

5.3.1 Process linearization

To design a controller for the system (25) we must first linearize it accodring to the discussions from section 5.1. Luckily, the same approach that we used for the rotors subsystem can be applied here as well. If we denote

$$v_v = \frac{1}{J_v}\left[g(A\cos\alpha_v - C\sin\alpha_v) + l_m F_v(\omega_v) - \frac{\Omega_h^2}{2}L\sin(2\alpha_v)\right] \tag{26}$$

the **vertical virtual control signal** (equal to the entire right side of the first equation) and

$$v_h = \frac{F_h(\omega_h) l_t \cos\alpha_v}{D\cos^2\alpha_v + E\sin^2\alpha_v + F} \tag{27}$$

the **horizontal virtual control signal** (the entire right side of the third equation), we obtain the linearized process

$$\begin{cases} \dfrac{\mathrm{d}\Omega_v}{\mathrm{d}t} = v_v \\ \dfrac{\mathrm{d}\alpha_v}{\mathrm{d}t} = \Omega_v \\ \dfrac{\mathrm{d}\Omega_h}{\mathrm{d}t} = v_h \\ \dfrac{\mathrm{d}\alpha_h}{\mathrm{d}t} = \Omega_h \end{cases} \tag{28}$$

having the same states and outputs as the non-linearized one, but the virtual input (control) signals $\mathbf{v} = \begin{bmatrix} v_v & v_h \end{bmatrix}^\top$. This respects the prerequisites of the feedback linearization technique because (28) is linear and the equations of the virtual control signals (26) and (27) are algebraic.

Moreover, except that the formulae of the virtual control signals, with respect to the real control signals, are different, the linearized dynamic of the vertical angle and speed is identical to the horizontal one. Therefore, we will discuss the controller design for the generalized dynamics, which will be then applied to both the vertical and horizontal controls.

5.3.2 Controller design

We have the generic system resulted from (28)

$$\begin{cases} \dfrac{\mathrm{d}\Omega}{\mathrm{d}t} = v \\ \dfrac{\mathrm{d}\alpha}{\mathrm{d}t} = \Omega \end{cases}$$

which by substitution gives

$$\frac{\mathrm{d}^2\alpha}{\mathrm{d}t^2} = v \Longrightarrow \alpha(t) = \int_0^t \left(\int_0^{\tau_1} v(\tau_2)\mathrm{d}\tau_2 \right) \mathrm{d}\tau_1 \tag{29}$$

which is a double integrator.

We have earlier proved (Theorem 5.1) that no matter the fact that the process contains two integrators, we must also have an integrator in the controller for the steady-state error to be reduced to zero. Since we have a derivative of order 2 in the system dynamics, we impose that the error of the main subsystem follows

$$\frac{\mathrm{d}^2\varepsilon(t)}{\mathrm{d}t^2} + K_d\frac{\mathrm{d}\varepsilon(t)}{\mathrm{d}t} + K_p\varepsilon(t) + K_i\int_0^t \varepsilon(\tau)\mathrm{d}\tau = 0 \tag{30}$$

which, by knowing that

$$\frac{\mathrm{d}^2\varepsilon(t)}{\mathrm{d}t^2} = \frac{\mathrm{d}^2}{\mathrm{d}t^2}\left(\alpha_{\text{ref}}(t) - \alpha(t)\right) = \frac{\mathrm{d}^2\alpha_{\text{ref}}(t)}{\mathrm{d}t^2} - v(t)$$

where $\alpha_{\text{ref}}(t)$ is the angle reference and $v(t)$ the virtual control signal (horizontal or vertical), transforms into

$$\frac{\mathrm{d}^2\alpha_{\text{ref}}(t)}{\mathrm{d}t^2} - v(t) + K_d\frac{\mathrm{d}\varepsilon(t)}{\mathrm{d}t} + K_p\varepsilon(t) + K_i\int_0^t \varepsilon(\tau)\mathrm{d}\tau = 0$$

which by rearranging gives us the control law of the linear controller

$$v(t) = \frac{\mathrm{d}^2\alpha_{\text{ref}}(t)}{\mathrm{d}t^2} + K_d\frac{\mathrm{d}\varepsilon(t)}{\mathrm{d}t} + K_p\varepsilon(t) + K_i\int_0^t \varepsilon(\tau)\mathrm{d}\tau \tag{31}$$

which is a PID (proportional-integral-derivative) controller *with feedforward*.

Now let us see how we can impose performance specifications on this controller. Take (30) and differentiate it once more to obtain a differential equation

$$\frac{\mathrm{d}^3\varepsilon(t)}{\mathrm{d}t^3} + K_d\frac{\mathrm{d}^2\varepsilon(t)}{\mathrm{d}t^2} + K_p\frac{\mathrm{d}\varepsilon(t)}{\mathrm{d}t} + K_i\varepsilon(t) = 0$$

This has the characteristic equation

$$\lambda^3 + K_d\lambda^2 + K_p\lambda + K_i = 0 \tag{32}$$

which describes the behavior of a standard third-order system. One of the roots is necessarily real; the rest is a second-order which can have complex conjugated or real roots. In its most general form, it can be arranged as

$$(\lambda - p_1)(\lambda^2 + 2\zeta\omega_n\lambda + \omega_n^2) = 0 \tag{33}$$

where p_1 is the real pole.

We can impose, as we did for the rotors, performance specifications in (33) and then tune the controller parameters K_d, K_p and K_i which result from the rearrangement of (33) like (32)

$$\begin{cases} K_d = 2\zeta\omega_n - p_1 \\ K_p = \omega_n^2 - 2\zeta\omega_n p_1 \\ K_i = -p_1\omega_n^2 \end{cases} \tag{34}$$

5.3.3 Steady state error discussion

Given the control law (31) that we have achieved, its Laplace transform is

$$V(s) = s^2R(s) + K_d sE(s) + K_pE(s) + K_i\frac{1}{s}E(s)$$

from which the controller transfer function is

$$C(s) = \frac{V(s)}{E(s)} = K_i\frac{1}{s} + K_p + K_d s + s^2\frac{R(s)}{E(s)}$$

which is of the form (20) if we denote $M(s) = K_p + K_d s + s^2\frac{R(s)}{E(s)}$. Theorem 5.1 ensures that the steady state error is zero.

5.4 Final control structure

It is now time to put together everything that we have constructed above. Since we have separated the system into two subsystems, this will now result in a nested control structure with a main and an inner control loop. Figure 10 presents the final control structure.

Let us study this control structure bit by bit. The upper part is the *horizontal control*, meaning the subpart responsible for controlling the azimuth angle α_h. The lower part is the *vertical control*, which controls the pitch angle α_v. Because the two are so similar, we will detail only the vertical control.

The signal that enters from the left is $\alpha_{\text{ref},v}$, the reference of the pitch. Both the reference and error (which is the reference minus the measured pitch) enter block 5, which is the linear main controller, that based on equation (31) computes the vertical virtual control signal v_v.

The virtual control signal v_v of the main subsystem then enters block 6 which is the inversion (cancelation) of the nonlinear part of the main subsystem. This block computes the real control signal of the main subsystem. But which is the control signal of this subsystem? Let us remember the discussion in section 5.3 where we stated that the input into the process, which is equivalent to the real control signal, is ω. Therefore, the output of this block is exactly the desired rpm of the main rotor. And because we have a nested structure, this rpm is actually a reference for the inner loop controller, $\omega_{\text{ref},v}$. But before moving on to that, it is worth noting that this block also needs the values of other physical quantities in the system in order to compute the real control signal $\omega_{\text{ref},v}$, and those are (looking at equation (26)) α_v, the pitch angle, and Ω_h, the azimuth speed. For the pitch angle it is very simple, the measured value is fed into the block. But the azimuth speed Ω_h is not measured. The angle α_h is measured, however, which is differentiated by the controller itself, from which an estimation of the real azimuth speed is obtained, which is fed into the block. For simplicity, the same notation Ω_h was used, but one must keep in mind that this is only an estimation of the real value, based on its primitive function α_h.

Next, the (inner) controller of the rotor follows. The reference (desired) rpm $\omega_{\text{ref},v}$ dictated by the main controller is fed into block 7, along with the error. Based on equation (24), the control law of the linear controller of the rotor subsystem, the virtual control signal of the rotor subsystem $v_{\text{rotor},v}$ is computed.

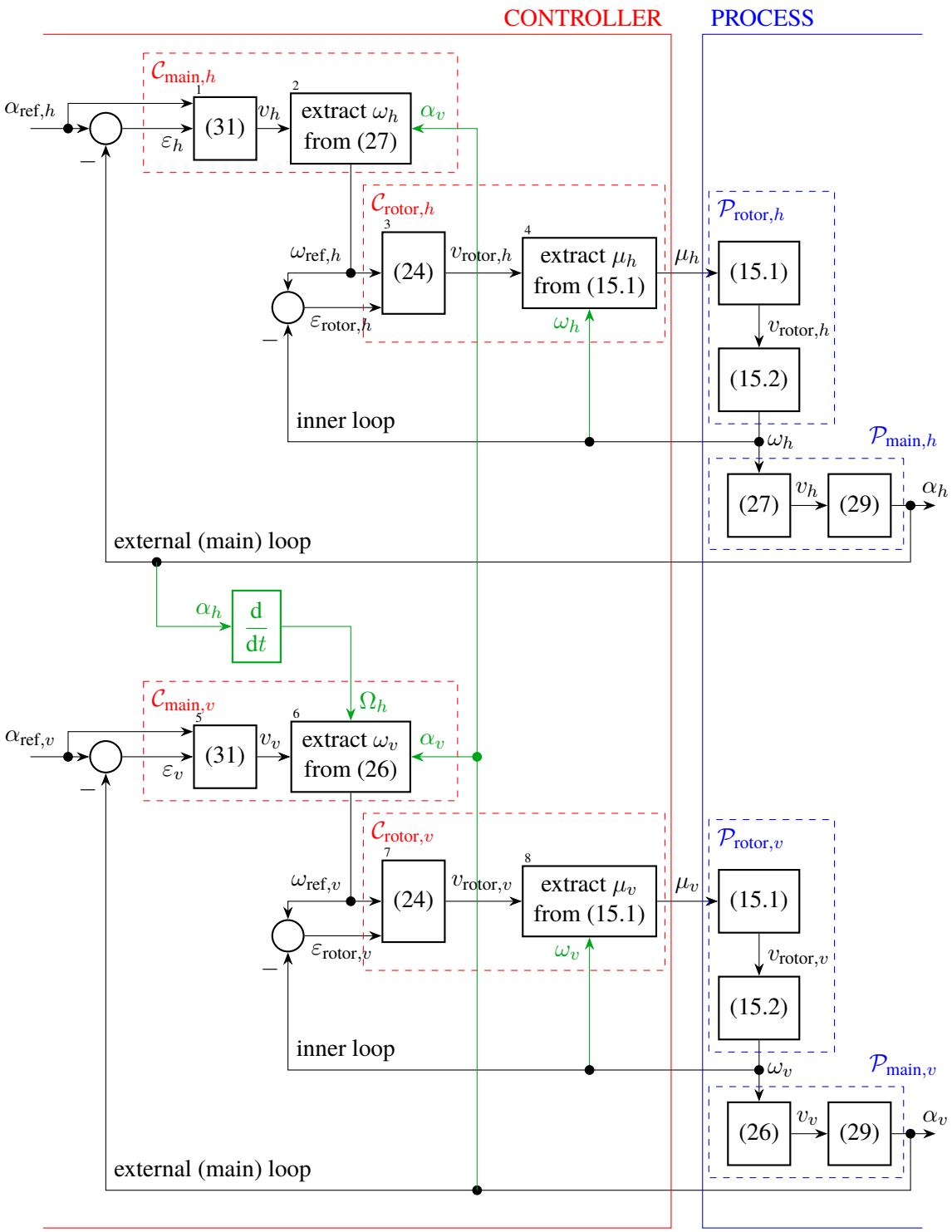


Figure 10: The final control structure.

The virtual control signal $v_{\text{rotor},v}$ is then turned into the real control signal of the rotor subsystem, μ_v, in block 8, which represents the inversion (cancelation) of the nonlinear part of the rotor subsystem described by (15.1). For this computation to be performed, the controller also needs to know the current value of ω_v, which is why the green dependency was drawn.

Then, the process follows. Its structure is not a matter of our choice anymore, this is dictated by the physics. So, firstly, the rotor subsystem $\mathcal{P}_{\text{rotor},v}$ is composed of the nonlinear algebraic (first) part (15.1) and then the linear (second) part (15.2). The output of the rotor subsystem is the rpm of the rotor, ω_v, which is measured and transmitted to the controller. Then comes the main subsystem $\mathcal{P}_{\text{main},v}$, whose input is exactly the rpm of the rotor, and which is composed of the nonlinear algebraic (first) part (26) and the linear (second) part (29). The output of the entire system is the pitch angle α_v, which is measured and transmitted to the controller.

The horizontal controller is very similar to the vertical one. The working principle is the same, only some of the equations are different and also less interdependences (the green arrows) are involved.

On this diagram it can also easily be seen which are the signals that are sent between the controller and the process: the controller provides μ_v and μ_h to the process, while the sensors in the process measure and provide α_v, ω_v, α_h and ω_h to the controller.

6 Implementation

The implementation of this project in MATLAB & Simulink has been done in strict accordance with section 5 and the resulted Simulink block diagram is the exact mirror of Figure 10 (only its side labelled "CONTROLLER", of course).

The entire project is publicly available at https://github.com/CodrinBerinde/TRAS_controller and therefore all the details will not be discussed here.

The inversion of the polynomials $F_v(\omega_v)$ and $F_h(\omega_h)$, involved in blocks 2 and 6, was implemented using lookup tables, in order to eliminate the need of solving 5th order polynomials at every time sample – a computation that is out of question. The lookup table works as a vector storing the values of the polynomial evaluated in equally-spaced points. When an interrogation is performed, a binary search is performed on the vector and the closest value of ω is retrieved for the interrogated value F of the polynomial.

There are a total of 4 linear controllers in this scheme. Two are for the main subsystems (horizontal and vertical) and two are for the rotors (main and tail). The parameters have been set identical for both rotor controllers. Through trial-and-error, intuition, and careful observation of the response of the process (because no other methods are available for this), the following specifications were imposed: for the main vertical control

$$\begin{cases} t_{s,v} = 3.5\,\text{s} \\ \zeta_v = 1.15 \end{cases}$$

and for the main horizontal control

$$\begin{cases} t_{s,h} = 3.7\,\text{s} \\ \zeta_h = 1.3 \end{cases}$$

The poles have been set at $p_{1,v} = 3.5\omega_{n,v}$ and $p_{1,h} = 3.5\omega_{n,h}$.

For the controls of the rotors we have set

$$\begin{cases} t_{s,\text{rotor}} = 1\,\text{s} \\ \zeta_{\text{rotor}} = 0.9 \end{cases}$$

The parameters K_i, K_p and, where it was the case, K_d of the controllers have been calculated from these specifications according to the appropriate formulas (22) and (34).

It must be noted that even though the damping coefficients ζ_v and ζ_h of the main controllers have been set supraunitary, oscillations still appear in the system due to inaccuracies in the modeling.

7 Performances

To exemplify the performance of our controller on the real-world device, three tests have been performed.

7.1 Zero angle references

The first one involves a zero angle reference for both azimuth and pitch. Results can be seen in Figure 11. First of all, we can see that the measured pitch angle starts from approximately $-0.5\,\text{rad} \approx -30°$ which is the resting position of the device. The behavior also presents an overshoot, even if we have set $\zeta_v > 1$. This happens due to imperfections in the modeling, which, as we can see, affect the behavior to a noticeably large extent. At the same time, on the azimuth, we can see that even though the reference is below the initially measured value, the angle increases. It later recovers and tends to zero, but this happens exactly due to the coupling between the two motions, which was described (and ignored in the modeling) in section 3: because initially the main rotor violently rotates clockwise, it creates a momentum in the counterclockwise (positive) movement direction of the azimuth angle. The horizontal controller will then compensate and gradually bring the azimuth to zero. The settling time of the pitch can be measured from the figure and it results in approximately 3 s which is in good accordance with the imposed value $t_{s,v}$.

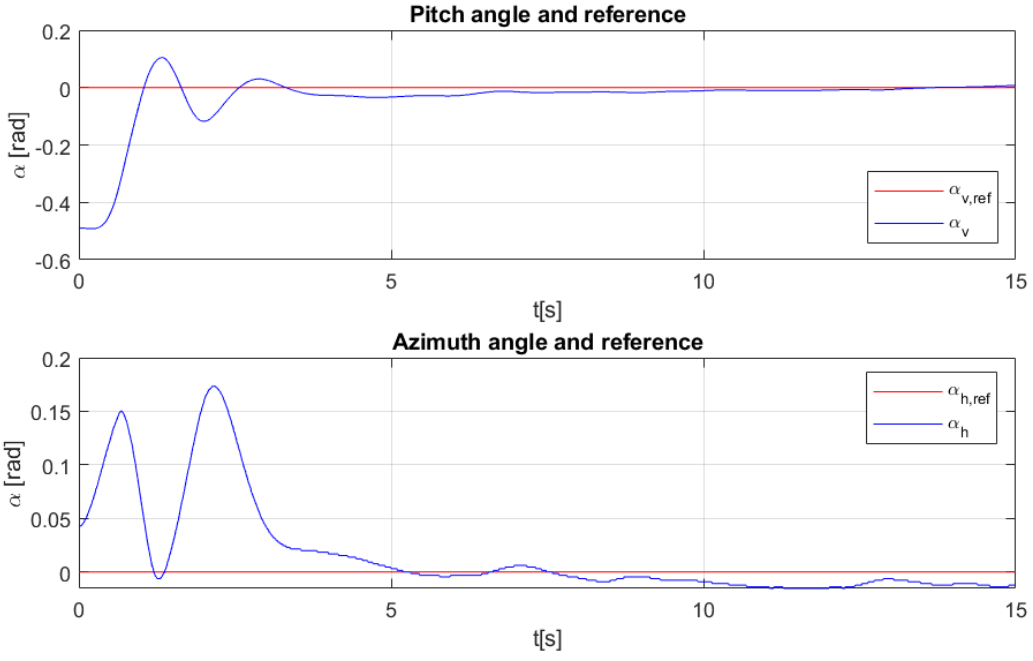


Figure 11: Test 1: performance of the controlled device for zero angle references

7.2 Zero angle references with external disturbances

The second test which has been performed is also with zero angle references for both azimuth and pitch, but this time external disturbances have been applied, namely giving a flick to the arm of the device with the hand. Disturbances have been applied on both planes and for each plane in both directions, giving a total of 4 disturbances. The results are presented in Figure 12.

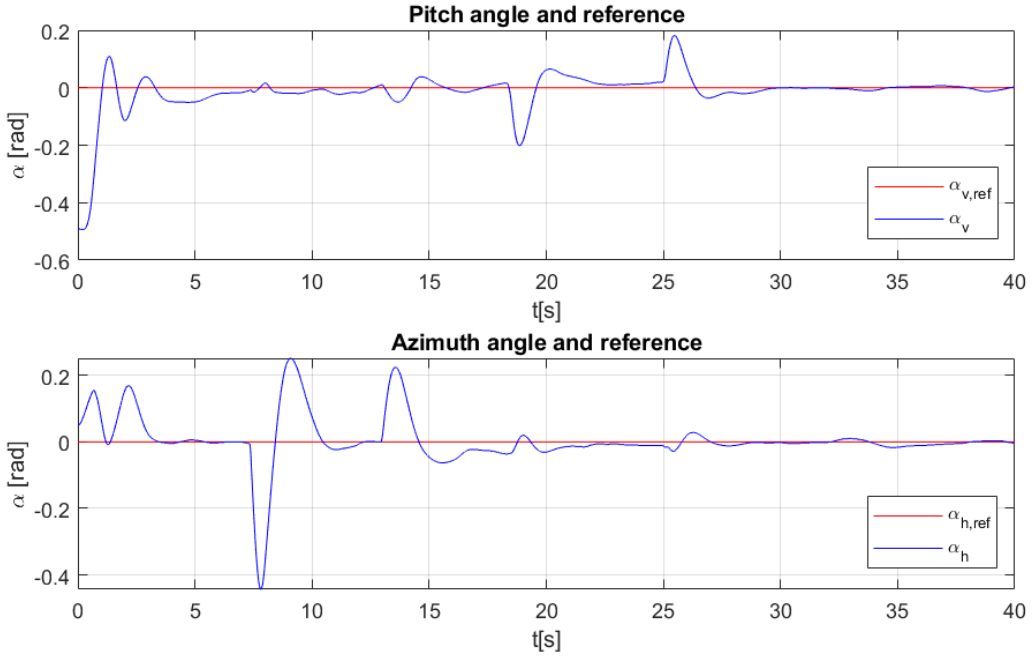


Figure 12: Test 2: performance of the controlled device for zero angle references – with disturbances

7.3 Circles in space

The last test which is presented here is with sinusoidal signals for both angle and pitch. Both have the same frequency $f = 0.12\,\text{Hz}$ and same amplitude $A = 30^\circ \approx 0.52\,\text{rad}$, but the azimuth has a phase shift of 90° with respect to the pitch, such that the tip of the robot arm performs circles in space. The results can be seen in Figure 13.

8 Conclusions

The way this project was presented through the current documentation, especially in the theoretical parts, may seem just a series of coincidences and fortunate events that have allowed us to somehow every time miraculously avoid getting stuck at a differential equation that is impossible to get over and that would have forced us to abandon everything and just make the roughest approximation ever, around some operating point, and just pray that the device does not go unstable the very instant it goes outside $\pm 5^\circ$ of that operating point. To some extent, it is true (about the series of lucky events). But for the most part, it was the attentive study that enabled us to design such a performant controller which – yes – gives the TRAS the ability to be stable and operate with decent performances at *any* reachable point on the sphere on which it can constructively rotate.

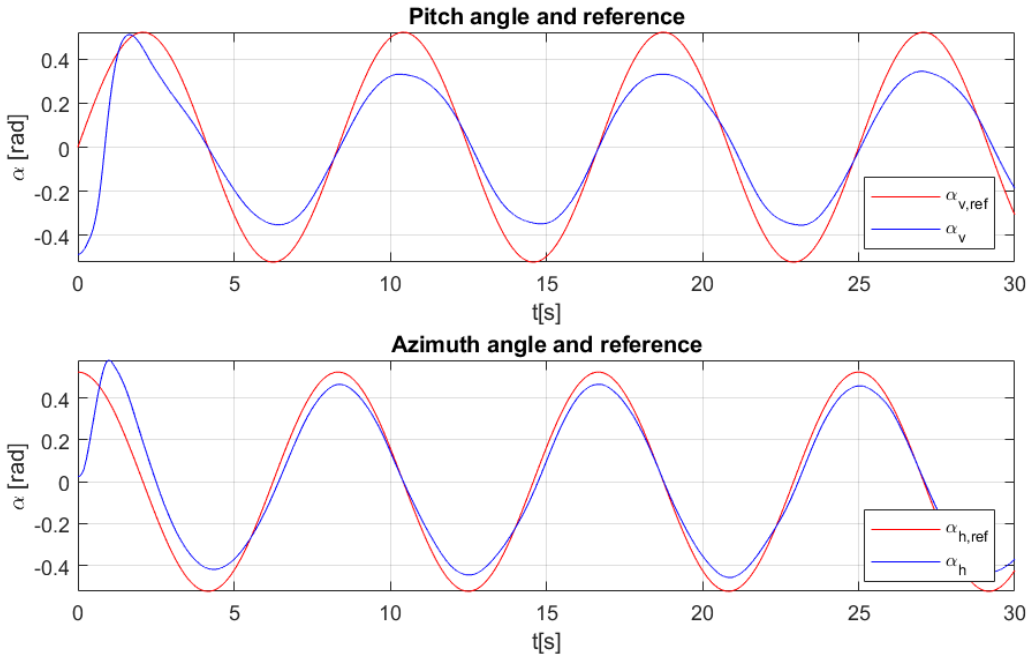


Figure 13: Test 3: performance of the controlled device for sinusoidal references – circles in space

The project has been presented like this, though, with delightful proofs and results here and there, mainly to show the beauty of control engineering.

References

[1] Jean-Jacques E. Slotine, Weiping Li. *Applied Nonlinear Control*. Prentice Hall, 1991.

[2] Richard C. Dorf, Robert H. Bishop. *Modern Control Systems*, 13th Edition. Pearson, 2016.